Supplementary Analysis: Poisson Regression and Diurnal Time Structure

ULB Credit Card Fraud Dataset

Arjun Sekhar

2026-08-14

1. Introduction

1.1. Objective

This notebook documents a supplementary analysis raised during supervisor review, examining whether the `Time` variable in the ULB dataset carries genuine temporal structure at the aggregate level, distinct from the instance-level survival structure examined in the main thesis notebook.

Two related questions are addressed:

1. Does fraud **rate** (rather than fraud classification) vary systematically across the observation window, using a Poisson/Negative Binomial regression framework with transaction count as an exposure offset?
2. Does the `Time` variable exhibit a recurring 24-hour (diurnal) cycle when folded onto a single day, consistent with the two-day structure of the dataset?

1.2. Motivation

This analysis complements the RSF-vs-LR comparison in the main notebook. That comparison evaluates whether individual-level survival structure (risk accumulation over time-to-event) benefits RSF over LR, and finds no advantage, consistent with the absence of genuine time-to-event structure in `Time`. This notebook asks a different, aggregate-level question: irrespective of instance-level survival structure, does the *rate* of fraud vary across time in a way that is model-agnostic and does not depend on RSF or LR being correctly specified?

If such structure exists, it would not contradict the main thesis finding, since a cyclical, exogenous rate pattern (e.g., driven by lower transaction volume and reduced monitoring overnight) is a different kind of temporal signal from the risk-accumulation structure RSF is designed to exploit. It would, however, sharpen the explanation for *why* `Time` was excluded from SMOTE interpolation and *why* the dataset lacks the structural conditions RSF benefits from.

2. Libraries and Data

```
## Step 1: Load required packages
## Note: MASS is loaded before dplyr to avoid the select() masking conflict
library(MASS)
library(dplyr)
library(ggplot2)
library(pander)

## Step 2: Set seed
set.seed(44940076)
```

```r
## Step 3: Load data
fraud_data <- read.csv('creditcard.csv', header = TRUE)
fraud_data <- na.omit(fraud_data)

dim(fraud_data)
```

```
## [1] 284807     31
```

```r
table(fraud_data$Class)
```

```
##
##      0      1
## 284315    492
```

3. Poisson Regression: Fraud Rate as a Function of Time

3.1. Bin Time into Fixed Windows

`Time` is binned into fixed-width windows, and within each window the fraud count and total transaction count are recorded. The transaction count serves as the exposure term (offset) in the regression, so the model is estimating fraud **rate per transaction**, not raw fraud count.

```
## Step 1: Set bin width and construct bins
bin_width <- 3600  # 1-hour bins (~48 bins across the ~48-hour window)

poisson_data <- fraud_data %>%
  mutate(time_bin = floor(Time / bin_width)) %>%
  group_by(time_bin) %>%
  summarise(
    fraud_count     = sum(Class),
    n_transactions  = n(),
    .groups = "drop"
  ) %>%
  mutate(
    bin_start_hr  = (time_bin * bin_width) / 3600,
    observed_rate = fraud_count / n_transactions
  )

pander(poisson_data, caption = "Fraud Counts and Exposure by Hourly Time Bin")
```

Table 1: Fraud Counts and Exposure by Hourly Time Bin

time_bin	fraud_count	n_transactions	bin_start_hr	observed_rate
0	2	3963	0	0.0005047
1	2	2217	1	0.0009021
2	21	1576	2	0.01332
3	13	1821	3	0.007139
4	6	1082	4	0.005545
5	11	1681	5	0.006544
6	3	1831	6	0.001638
7	23	3368	7	0.006829
8	5	5179	8	0.0009654
9	15	7878	9	0.001904
10	2	8288	10	0.0002413
11	43	8517	11	0.005049
12	9	7732	12	0.001164
13	9	7585	13	0.001187
14	13	8029	14	0.001619
15	14	7836	15	0.001787
16	14	7786	16	0.001798
17	12	7882	17	0.001522
18	15	8607	18	0.001743
19	7	7994	19	0.0008757
20	8	8980	20	0.0008909
21	14	9895	21	0.001415
22	3	8977	22	0.0003342
23	17	6082	23	0.002795
24	4	3732	24	0.001072
25	8	2003	25	0.003994

time_bin	fraud_count	n_transactions	bin_start_hr	observed_rate
26	36	1752	26	0.02055
27	4	1671	27	0.002394
28	17	1127	28	0.01508
29	0	1309	29	0
30	6	2270	30	0.002643
31	0	3875	31	0
32	4	5097	32	0.0007848
33	1	7960	33	0.0001256
34	6	8310	34	0.000722
35	10	8339	35	0.001199
36	8	7688	36	0.001041
37	8	7780	37	0.001028
38	10	8541	38	0.001171
39	12	8625	39	0.001391
40	8	8667	40	0.000923
41	17	8284	41	0.002052
42	18	8432	42	0.002135
43	12	7655	43	0.001568
44	10	7776	44	0.001286
45	2	7808	45	0.0002561
46	6	6464	46	0.0009282
47	4	4856	47	0.0008237

3.2. Fit Poisson GLM

```
## Step 1: Fit Poisson model with log(n_transactions) as offset
poisson_model <- glm(
  fraud_count ~ time_bin,
  offset = log(n_transactions),
  family = poisson,
  data   = poisson_data
)

summary(poisson_model)
```

```
##
## Call:
## glm(formula = fraud_count ~ time_bin, family = poisson, data = poisson_data,
##     offset = log(n_transactions))
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.81702    0.08903 -65.334  < 2e-16 ***
## time_bin    -0.02279    0.00350  -6.511 7.46e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 452.15  on 47  degrees of freedom
## Residual deviance: 408.69  on 46  degrees of freedom
## AIC: 594.73
```

```
##
## Number of Fisher Scoring iterations: 5
```

3.3. Overdispersion Check

```
## Step 1: Compute Pearson dispersion ratio
dispersion_ratio <- sum(residuals(poisson_model, type = "pearson")^2) / poisson_model$df.residual
dispersion_ratio
```

```
## [1] 16.74085
```

Observations: The dispersion ratio of 16.74 indicates substantial overdispersion relative to the Poisson assumption of a ratio near 1. This means the standard errors and p-values from the plain Poisson model are unreliable and understate the true uncertainty. Quasi-Poisson and Negative Binomial specifications are therefore fitted as corrections in Section 3.4.

3.4. Robustness: Quasi-Poisson and Negative Binomial

```
## Step 1: Quasi-Poisson (rescales SEs by the dispersion ratio, same point estimate)
quasipoisson_model <- glm(
  fraud_count ~ time_bin,
  offset = log(n_transactions),
  family = quasipoisson,
  data   = poisson_data
)
summary(quasipoisson_model)
```

```
##
## Call:
## glm(formula = fraud_count ~ time_bin, family = quasipoisson,
##     data = poisson_data, offset = log(n_transactions))
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -5.81702    0.36429 -15.968   <2e-16 ***
## time_bin    -0.02279    0.01432  -1.591    0.118
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasipoisson family taken to be 16.74093)
##
##     Null deviance: 452.15  on 47  degrees of freedom
## Residual deviance: 408.69  on 46  degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 5
```

```
## Step 2: Negative Binomial (explicitly models the mean-variance relationship)
nb_model <- glm.nb(
  fraud_count ~ time_bin + offset(log(n_transactions)),
  data = poisson_data
)
summary(nb_model)
```

```
##
```

```
## Call:
## glm.nb(formula = fraud_count ~ time_bin + offset(log(n_transactions)),
##     data = poisson_data, init.theta = 1.150858842, link = log)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.37947    0.27846 -19.319  < 2e-16 ***
## time_bin    -0.02787    0.01025  -2.719  0.00655 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(1.1509) family taken to be 1)
##
##     Null deviance: 62.212  on 47  degrees of freedom
## Residual deviance: 55.639  on 46  degrees of freedom
## AIC: 346.8
##
## Number of Fisher Scoring iterations: 1
##
##
##               Theta:  1.151
##           Std. Err.:  0.251
##
##  2 x log-likelihood:  -340.802
```

3.5. Null Model Comparison (Linear Trend)

```
## Step 1: Fit intercept-only null model
null_model <- glm(
  fraud_count ~ 1,
  offset = log(n_transactions),
  family = poisson,
  data   = poisson_data
)

## Step 2: Compare via likelihood ratio test and AIC
anova(null_model, poisson_model, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: fraud_count ~ 1
## Model 2: fraud_count ~ time_bin
##   Resid. Df Resid. Dev Df Deviance  Pr(>Chi)
## 1        47     452.15
## 2        46     408.69  1   43.457 4.333e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

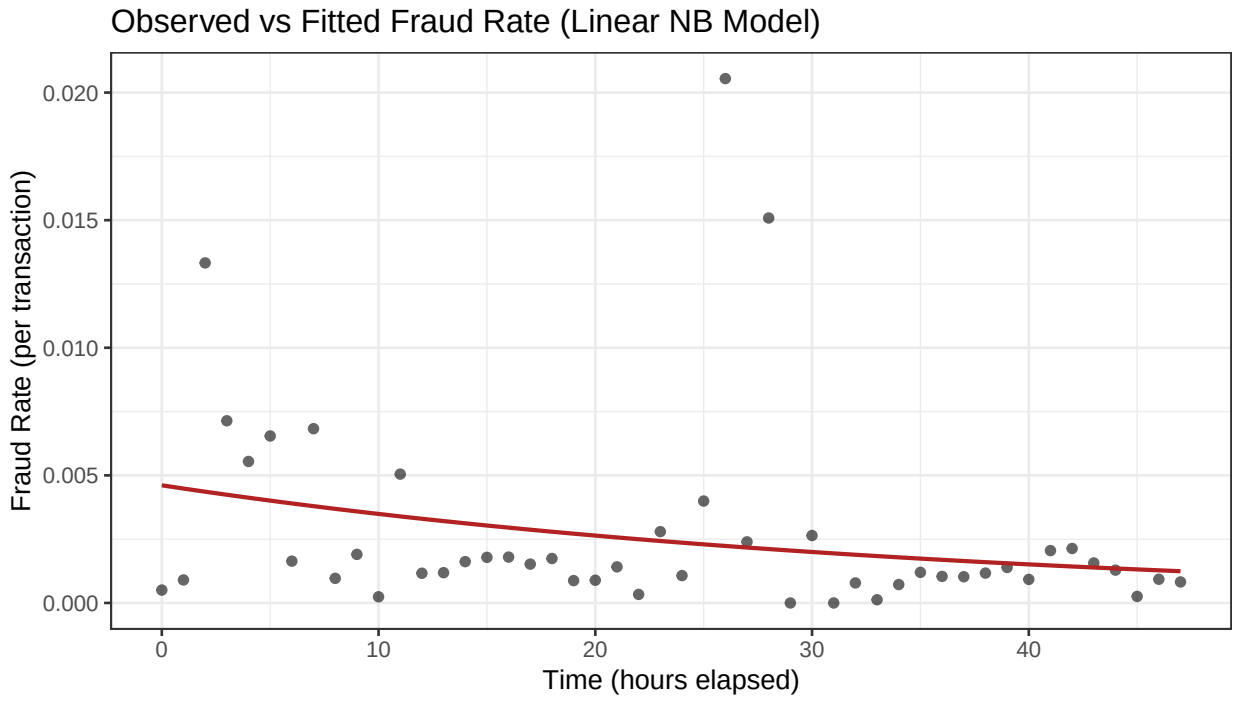
```
AIC(null_model, poisson_model, nb_model)
```

```
##               df      AIC
## null_model     1 636.1887
## poisson_model  2 594.7312
## nb_model       3 346.8022
```

Observations: The plain Poisson model shows a significant negative linear trend in `time_bin` ($p < 0.001$), but this result is unreliable given the overdispersion identified in Section 3.3. Under the quasi-Poisson correction, which inflates standard errors by the dispersion ratio, the same coefficient is no longer significant ($p = 0.118$). The Negative Binomial model, which explicitly models overdispersion via maximum likelihood rather than a flat SE rescaling, retains a significant negative trend ($p = 0.007$) and fits substantially better than both the null and plain Poisson models by AIC. This divergence between the two overdispersion corrections motivated a closer look at the shape of the trend, rather than accepting the linear specification at face value.

```
## Step 3: Visualise observed vs fitted rate under the linear NB model
poisson_data$fitted_rate_linear <- predict(nb_model, type = "response") / poisson_data$n_transactions

ggplot(poisson_data, aes(x = bin_start_hr)) +
  geom_point(aes(y = observed_rate), alpha = 0.6) +
  geom_line(aes(y = fitted_rate_linear), colour = "firebrick", linewidth = 0.8) +
  labs(
    title = "Observed vs Fitted Fraud Rate (Linear NB Model)",
    x = "Time (hours elapsed)",
    y = "Fraud Rate (per transaction)"
  ) +
  theme_bw()
```



The plot reveals two prominent elevated bins around hours 26 and 28 (both falling in the second overnight period), which appear to be driving much of the fitted linear decline. This is consistent with a cyclical rather than monotonic pattern, examined formally in Section 3.6.

3.6. Harmonic (Diurnal Cycle) Poisson Regression

Rather than a monotonic linear trend, a 24-hour harmonic term more directly tests whether fraud rate follows a diurnal cycle, consistent with the two elevated overnight bins observed in Section 3.5.

```
## Step 1: Add sin/cos harmonic terms based on a 24-hour period
poisson_data <- poisson_data %>%
  mutate(
    hour_angle = 2 * pi * bin_start_hr / 24,
    sin_24     = sin(hour_angle),
    cos_24     = cos(hour_angle)
  )

## Step 2: Fit NB model with harmonic terms
nb_harmonic_model <- glm.nb(
  fraud_count ~ sin_24 + cos_24 + offset(log(n_transactions)),
  data = poisson_data
)

summary(nb_harmonic_model)
```

```
##
## Call:
## glm.nb(formula = fraud_count ~ sin_24 + cos_24 + offset(log(n_transactions)),
##     data = poisson_data, init.theta = 1.40995654, link = log)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -6.0996     0.1294 -47.127  < 2e-16 ***
## sin_24        0.5746     0.1838   3.126  0.00177 **
## cos_24        0.4640     0.1823   2.544  0.01095 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(1.41) family taken to be 1)
##
##     Null deviance: 73.346  on 47  degrees of freedom
## Residual deviance: 53.829  on 45  degrees of freedom
## AIC: 338.05
##
## Number of Fisher Scoring iterations: 1
##
##
##               Theta:  1.410
##           Std. Err.:  0.317
##
##  2 x log-likelihood:  -330.047
```

```
## Step 3: Compare null, linear, and harmonic NB models via AIC
AIC(null_model, nb_model, nb_harmonic_model)
```

```
##                   df      AIC
## null_model         1 636.1887
## nb_model           3 346.8022
## nb_harmonic_model  4 338.0472
```

```r
## Step 4: Likelihood ratio test - harmonic model vs null
null_nb_model <- glm.nb(
  fraud_count ~ offset(log(n_transactions)),
  data = poisson_data
)

anova(null_nb_model, nb_harmonic_model)
```

```
## Likelihood ratio tests of Negative Binomial Models
##
## Response: fraud_count
##                                            Model    theta Resid. df
## 1                    offset(log(n_transactions)) 1.009778        47
## 2 sin_24 + cos_24 + offset(log(n_transactions)) 1.409957        45
##      2 x log-lik.   Test    df LR stat.      Pr(Chi)
## 1       -346.9867
## 2       -330.0472 1 vs 2     2 16.93951 0.0002097158
```

3.7. Peak Timing and Amplitude

```r
## Step 1: Extract amplitude and phase from the harmonic coefficients
harmonic_coefs <- coef(nb_harmonic_model)
A <- harmonic_coefs["sin_24"]
B <- harmonic_coefs["cos_24"]

amplitude        <- unname(sqrt(A^2 + B^2))
peak_hour_of_day <- unname((atan2(B, A) * 24) / (2 * pi))
peak_hour_of_day <- (peak_hour_of_day + 24) %% 24

amplitude
```

```
## [1] 0.7385143
```

```r
peak_hour_of_day
```

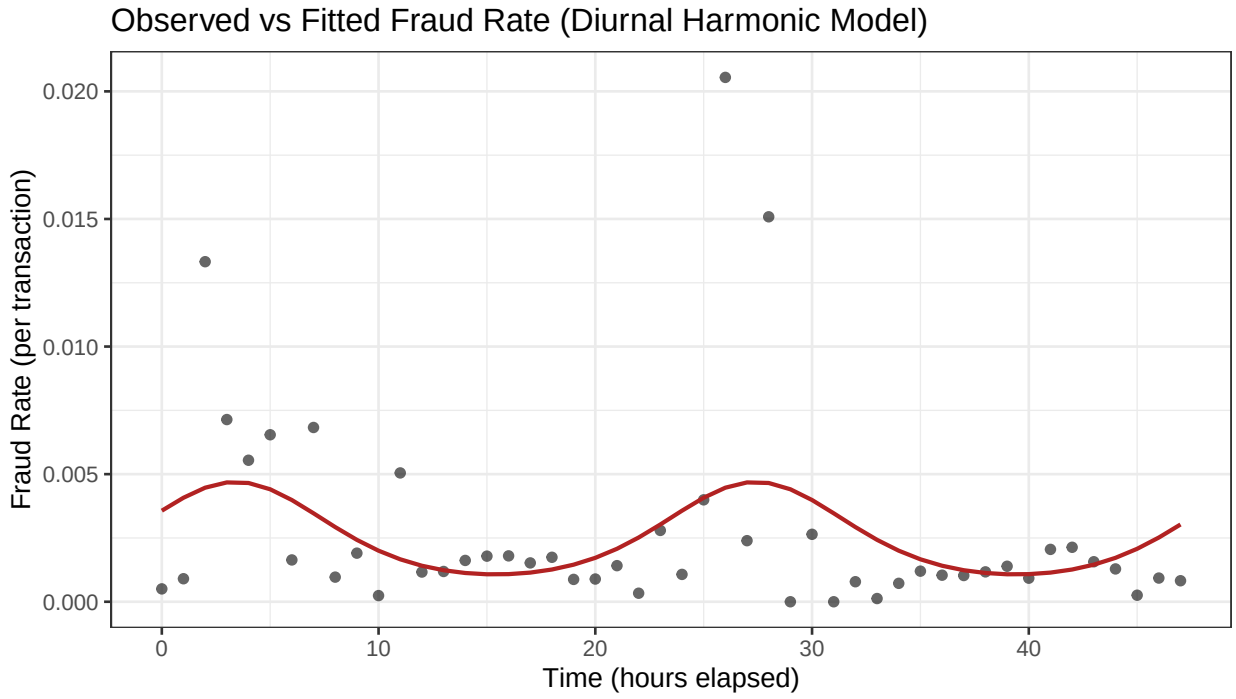
```
## [1] 2.594652
```

Observations: The harmonic model improves substantially on both the null model (AIC 636.2) and the linear trend model (AIC 346.8), reaching an AIC of 338, and the likelihood ratio test confirms the two harmonic terms jointly add significant explanatory power beyond the null ($p < 0.001$). The fitted cycle peaks at approximately 2:36 each day, with fraud rate at peak running roughly 2.1x the daily mean level. This is consistent with the relative fraud-risk pattern already documented in the main notebook (Section 6.6), which showed relative risk peaking 3-3.5x during low-activity overnight periods, now confirmed via an independent, model-based approach on the actual `Time` variable.

```r
## Step 2: Visualise fitted diurnal curve against observed rate
poisson_data$fitted_rate_harmonic <- predict(nb_harmonic_model, type = "response") / poisson_data$n_tra

ggplot(poisson_data, aes(x = bin_start_hr)) +
  geom_point(aes(y = observed_rate), alpha = 0.6) +
  geom_line(aes(y = fitted_rate_harmonic), colour = "firebrick", linewidth = 0.8) +
  labs(
    title = "Observed vs Fitted Fraud Rate (Diurnal Harmonic Model)",
    x = "Time (hours elapsed)",
    y = "Fraud Rate (per transaction)"
  ) +
```

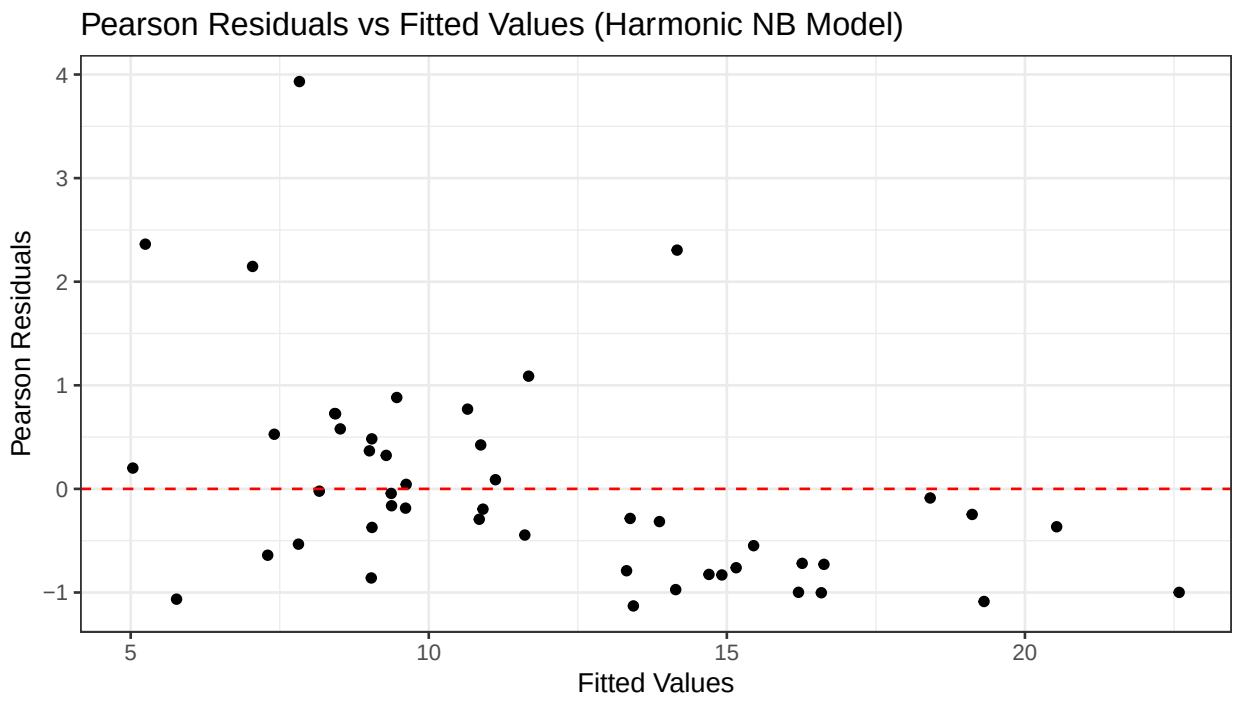
```
  theme_bw()
```



3.8. Residual Diagnostics

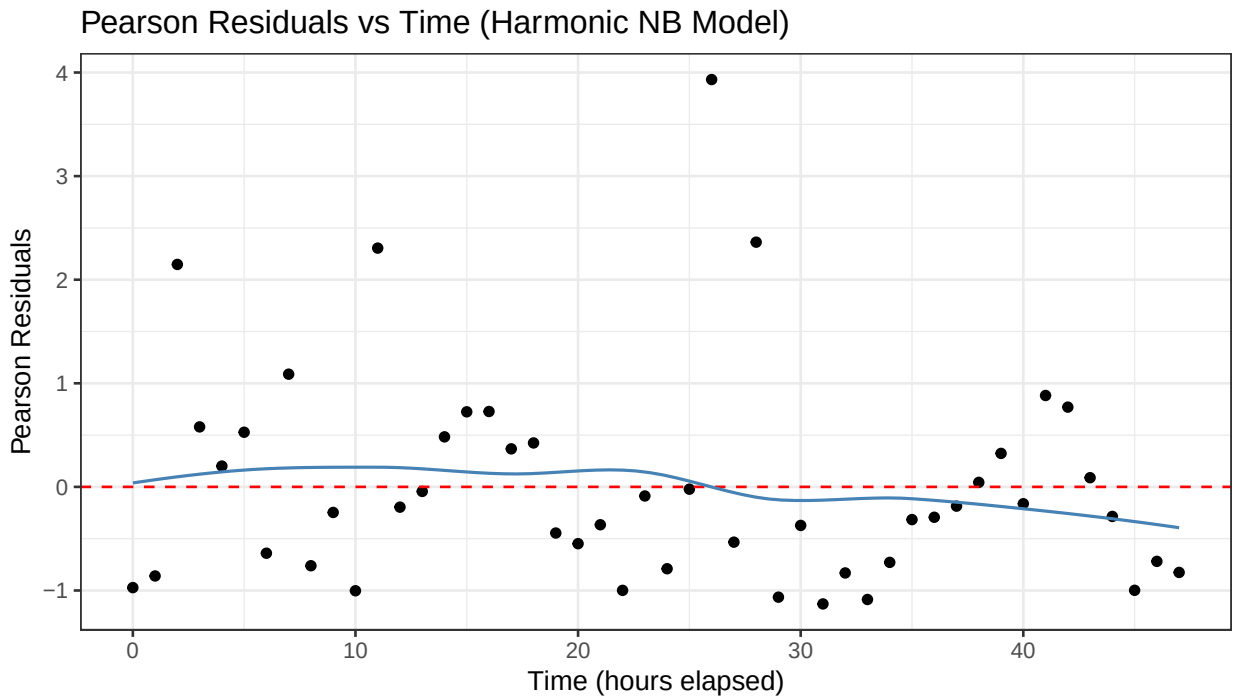
```
## Step 1: Pearson residuals vs fitted
poisson_data$resid_pearson <- residuals(nb_harmonic_model, type = "pearson")
poisson_data$fitted_vals   <- fitted(nb_harmonic_model)

ggplot(poisson_data, aes(x = fitted_vals, y = resid_pearson)) +
  geom_point() +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  labs(
    title = "Pearson Residuals vs Fitted Values (Harmonic NB Model)",
    x = "Fitted Values",
    y = "Pearson Residuals"
  ) +
  theme_bw()
```




```
## Step 2: Residuals vs time
ggplot(poisson_data, aes(x = bin_start_hr, y = resid_pearson)) +
  geom_point() +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  geom_smooth(se = FALSE, colour = "steelblue", linewidth = 0.6) +
  labs(
    title = "Pearson Residuals vs Time (Harmonic NB Model)",
    x = "Time (hours elapsed)",
    y = "Pearson Residuals"
  ) +
  theme_bw()
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
## Step 3: Refitted dispersion ratio
dispersion_ratio_harmonic <- sum(poisson_data$resid_pearson^2) / nb_harmonic_model$df.residual
dispersion_ratio_harmonic
```

```
## [1] 1.104307
```

```
## Step 4: Bins with large residuals (|resid| > 2)
poisson_data %>%
  filter(abs(resid_pearson) > 2) %>%
  select(time_bin, bin_start_hr, fraud_count, n_transactions, observed_rate,
         fitted_rate_harmonic, resid_pearson)
```

```
## # A tibble: 4 x 7
##   time_bin bin_start_hr fraud_count n_transactions observed_rate
##      <dbl>        <dbl>       <int>          <int>         <dbl>
## 1        2            2          21           1576       0.0133
## 2       11           11          43           8517       0.00505
## 3       26           26          36           1752       0.0205
## 4       28           28          17           1127       0.0151
## # i 2 more variables: fitted_rate_harmonic <dbl>, resid_pearson <dbl>
```

Observations: The dispersion ratio falls from 16.7 under the plain Poisson model to 1.1 under the harmonic Negative Binomial model, and the residuals-vs-time plot shows no remaining systematic wave pattern, indicating the harmonic term captures the dominant cyclical structure. Four bins retain residuals exceeding 2 in absolute value, two of which (hours 2 and 26, exactly one 24-hour cycle apart) sit close to the fitted peak and are both underpredicted, suggesting the true overnight surge may be somewhat more concentrated than a single sinusoid can represent. Given the already strong fit (dispersion ratio near 1) and the risk of overfitting a 48-bin series with additional harmonic terms, this is reported as a noted limitation rather than grounds for a more complex model specification.

3.9. Full-Feature Poisson Regression with Harmonic Time (Individual-Level)

Sections 3.1-3.8 modelled fraud rate at the aggregate (hourly-bin) level, using time alone as the predictor. As a follow-up, Maurizio suggested extending this to the individual-transaction level, including the full feature set (V1-V28, Amount) as used in the Logistic Regression models in the main notebook, with raw/proxy `Time` replaced by the harmonic time-of-day terms (`sin_24`, `cos_24`).

Since `Class` is a binary outcome rather than a count, this specification is a "modified Poisson" (or log-binomial-style) regression (Zou, 2004): a Poisson model with a log link fitted to a binary outcome, used to obtain relative-risk-style coefficients rather than the odds ratios produced by logistic regression. Because the Poisson variance assumption (Var = Mean) does not hold for a Bernoulli outcome, robust (sandwich) standard errors are used for valid inference rather than the default `glm()` output.

```
fraud_data <- fraud_data %>%
  mutate(
    time_of_day_sec = Time %% 86400,
    time_of_day_hr  = time_of_day_sec / 3600,
    day_number      = floor(Time / 86400) + 1
  )
```

```
## Step 1: Add harmonic time terms to the individual-level dataset
set.seed(44940076)

fraud_data <- fraud_data %>%
  mutate(
    hour_angle = 2 * pi * time_of_day_hr / 24,
    sin_24     = sin(hour_angle),
    cos_24     = cos(hour_angle)
  )

## Step 2: Build the modelling dataset - same feature set as the LR models,
## with raw/proxy Time replaced by the harmonic terms
poisson_full_data <- fraud_data %>%
  dplyr::select(V1:V28, Amount, sin_24, cos_24, Class)

## Step 3: Fit Poisson ("modified Poisson" / log-binomial-style) regression
poisson_full_model <- glm(
  Class ~ .,
  family = poisson(link = "log"),
  data   = poisson_full_data
)

summary(poisson_full_model)
```

```
##
## Call:
## glm(formula = Class ~ ., family = poisson(link = "log"), data = poisson_full_data)
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -7.9586644  0.1273362 -62.501  < 2e-16 ***
## V1          -0.1264374  0.0268385  -4.711 2.46e-06 ***
## V2           0.2834022  0.0511409   5.542 3.00e-08 ***
## V3          -0.0012320  0.0356766  -0.035 0.972452
## V4           0.5411829  0.0680394   7.954 1.81e-15 ***
## V5           0.1398547  0.0595259   2.349 0.018800 *
```

```
## V6          -0.2757523  0.0643886  -4.283 1.85e-05 ***
## V7           0.2273888  0.0524590   4.335 1.46e-05 ***
## V8          -0.0848542  0.0226332  -3.749 0.000177 ***
## V9           0.1952420  0.0930098   2.099 0.035803 *
## V10         -0.2945126  0.0703418  -4.187 2.83e-05 ***
## V11         -0.0290396  0.0521012  -0.557 0.577275
## V12          0.3659857  0.0691112   5.296 1.19e-07 ***
## V13         -0.3134072  0.0653246  -4.798 1.61e-06 ***
## V14         -0.3953042  0.0424032  -9.323  < 2e-16 ***
## V15         -0.0905725  0.0626635  -1.445 0.148351
## V16          0.2073484  0.1174163   1.766 0.077408 .
## V17         -0.2940395  0.0530929  -5.538 3.06e-08 ***
## V18          0.2536146  0.1202798   2.109 0.034984 *
## V19          0.2721449  0.0807674   3.369 0.000753 ***
## V20          0.0535874  0.0504363   1.062 0.288019
## V21          0.1035892  0.0402344   2.575 0.010034 *
## V22          0.1674096  0.0893466   1.874 0.060970 .
## V23         -0.0810398  0.0422671  -1.917 0.055196 .
## V24          0.2066648  0.1124144   1.838 0.066001 .
## V25          0.0563461  0.0819861   0.687 0.491916
## V26         -0.4803621  0.1470256  -3.267 0.001086 **
## V27         -0.0086343  0.0391060  -0.221 0.825253
## V28          0.0702255  0.0502442   1.398 0.162208
## Amount       0.0003647  0.0002986   1.221 0.221959
## sin_24      -0.2559980  0.1109210  -2.308 0.021003 *
## cos_24      -0.0190852  0.0868868  -0.220 0.826139
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 6259.3  on 284806  degrees of freedom
## Residual deviance: 2780.6  on 284775  degrees of freedom
## AIC: 3828.6
##
## Number of Fisher Scoring iterations: 11
```

```
## Step 4: Robust (sandwich) standard errors
## Required because the Poisson variance assumption does not hold for a
## binary outcome; this gives valid inference while retaining the
## relative-risk interpretation of the coefficients.
library(sandwich)
library(lmtest)

coeftest(poisson_full_model, vcov = vcovHC(poisson_full_model, type = "HC0"))
```

```
##
## z test of coefficients:
##
##                Estimate  Std. Error  z value  Pr(>|z|)
## (Intercept) -7.95866442  0.12427464 -64.0409 < 2.2e-16 ***
## V1          -0.12643741  0.02593486  -4.8752 1.087e-06 ***
## V2           0.28340220  0.05161874   5.4903 4.013e-08 ***
## V3          -0.00123203  0.03538760  -0.0348 0.9722271
## V4           0.54118290  0.06714306   8.0601 7.620e-16 ***
```

```
## V5           0.13985469  0.05452338   2.5650 0.0103164 *
## V6          -0.27575226  0.04965908  -5.5529 2.810e-08 ***
## V7           0.22738882  0.04496102   5.0575 4.249e-07 ***
## V8          -0.08485423  0.02214991  -3.8309 0.0001277 ***
## V9           0.19524203  0.09226563   2.1161 0.0343375 *
## V10         -0.29451264  0.06878975  -4.2813 1.858e-05 ***
## V11         -0.02903957  0.05467680  -0.5311 0.5953403
## V12          0.36598567  0.07844391   4.6656 3.078e-06 ***
## V13         -0.31340716  0.08426574  -3.7193 0.0001998 ***
## V14         -0.39530422  0.04641744  -8.5163 < 2.2e-16 ***
## V15         -0.09057250  0.06792325  -1.3335 0.1823830
## V16          0.20734839  0.13477918   1.5384 0.1239434
## V17         -0.29403950  0.05612484  -5.2390 1.614e-07 ***
## V18          0.25361458  0.13062129   1.9416 0.0521853 .
## V19          0.27214485  0.09633517   2.8250 0.0047284 **
## V20          0.05358745  0.04909439   1.0915 0.2750447
## V21          0.10358919  0.04171214   2.4834 0.0130124 *
## V22          0.16740961  0.09247306   1.8104 0.0702398 .
## V23         -0.08103977  0.04075235  -1.9886 0.0467463 *
## V24          0.20666481  0.10695602   1.9322 0.0533298 .
## V25          0.05634608  0.09935002   0.5671 0.5706142
## V26         -0.48036213  0.16251297  -2.9558 0.0031182 **
## V27         -0.00863435  0.04004766  -0.2156 0.8292982
## V28          0.07022548  0.04307039   1.6305 0.1029998
## Amount       0.00036469  0.00025485   1.4310 0.1524320
## sin_24      -0.25599796  0.10624861  -2.4094 0.0159777 *
## cos_24      -0.01908519  0.08490180  -0.2248 0.8221416
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
## Step 5: Discrimination metrics, for comparison with the full-dataset
## LR results reported in the main notebook (Section 8.4.1)
library(pROC)
library(PRROC)

poisson_full_pred <- predict(poisson_full_model, type = "response")

## Sanity check: predicted probabilities should not meaningfully exceed 1
## under the log link, given the low base rate
max(poisson_full_pred)
```

```
## [1] 5.673826
```

```r
poisson_full_auc <- as.numeric(auc(
  roc(response = poisson_full_data$Class, predictor = poisson_full_pred, quiet = TRUE)
))

poisson_full_pr <- pr.curve(
  scores.class0 = poisson_full_pred[poisson_full_data$Class == 1],
  scores.class1 = poisson_full_pred[poisson_full_data$Class == 0],
  curve = FALSE
)

poisson_full_auc
```

```
## [1] 0.9625373
```

```
poisson_full_pr$auc.integral
```

```
## [1] 0.6299554
```

Observations: Under robust standard errors, `sin_24` remains marginally significant ($p = 0.016$) while `cos_24` does not ($p = 0.822$), which is a weaker and more mixed result than the time-only aggregate model in Section 3.6, where both harmonic terms were clearly significant. The maximum predicted probability of 5.67 exceeds 1, confirming that the log-link specification is not well-calibrated at the individual level for this dataset, most likely driven by the same strong separators (V14, V4) identified in the main notebook's variable importance analysis (Section 9.2); AUC and AUPRC remain valid as rank-based metrics despite this, but the coefficient estimates themselves should not be over-interpreted. Discrimination (AUC = 0.963, AUPRC = 0.63) is also notably weaker than the full-dataset Logistic Regression results reported in the main notebook (AUPRC 0.760-0.769, Section 8.4), suggesting the modified Poisson specification is a less effective classifier here than standard LR even before considering the calibration issue.

3.10. Joint Significance Test: Harmonic Terms Beyond the Full Feature Set

Since `sin_24` and `cos_24` jointly define the amplitude and phase of a diurnal cycle, individual-coefficient significance is not the most defensible way to assess whether the cycle adds anything once V1-V28 and Amount are already in the model. A joint test, dropping both harmonic terms together, gives a single answer to that question.

```
## Step 1: Fit the reduced model (drops sin_24 and cos_24)
poisson_reduced_model <- glm(
  Class ~ . - sin_24 - cos_24,
  family = poisson(link = "log"),
  data   = poisson_full_data
)

## Step 2: Standard likelihood ratio test
anova(poisson_reduced_model, poisson_full_model, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: Class ~ (V1 + V2 + V3 + V4 + V5 + V6 + V7 + V8 + V9 + V10 + V11 +
##     V12 + V13 + V14 + V15 + V16 + V17 + V18 + V19 + V20 + V21 +
##     V22 + V23 + V24 + V25 + V26 + V27 + V28 + Amount + sin_24 +
##     cos_24) - sin_24 - cos_24
## Model 2: Class ~ V1 + V2 + V3 + V4 + V5 + V6 + V7 + V8 + V9 + V10 + V11 +
##     V12 + V13 + V14 + V15 + V16 + V17 + V18 + V19 + V20 + V21 +
##     V22 + V23 + V24 + V25 + V26 + V27 + V28 + Amount + sin_24 +
##     cos_24
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1    284777     2786.5
## 2    284775     2780.6  2   5.9425  0.05124 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
## Step 3: Robust Wald test, consistent with the sandwich SEs used in Section 3.9
waldtest(
  poisson_reduced_model, poisson_full_model,
  vcov = vcovHC(poisson_full_model, type = "HC0"),
  test = "Chisq"
)
```

```
## Wald test
##
## Model 1: Class ~ (V1 + V2 + V3 + V4 + V5 + V6 + V7 + V8 + V9 + V10 + V11 +
##     V12 + V13 + V14 + V15 + V16 + V17 + V18 + V19 + V20 + V21 +
##     V22 + V23 + V24 + V25 + V26 + V27 + V28 + Amount + sin_24 +
##     cos_24) - sin_24 - cos_24
## Model 2: Class ~ V1 + V2 + V3 + V4 + V5 + V6 + V7 + V8 + V9 + V10 + V11 +
##     V12 + V13 + V14 + V15 + V16 + V17 + V18 + V19 + V20 + V21 +
##     V22 + V23 + V24 + V25 + V26 + V27 + V28 + Amount + sin_24 +
##     cos_24
##   Res.Df Df  Chisq Pr(>Chisq)
## 1 284777
## 2 284775  2 6.0961    0.04745 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Observations: The two tests sit on opposite sides of the conventional 0.05 threshold, though both are close to it: the standard likelihood ratio test gives p = 0.051, while the robust Wald test (which accounts for the same variance-structure violation addressed in Section 3.9) gives p = 0.047. Rather than treating this as a clear pass or fail, the honest characterisation is that the diurnal cycle adds, at most, marginal explanatory power once V1-V28 and Amount are already in the model - a materially weaker result than the aggregate-level test in Section 3.6, where the harmonic terms improved substantially on the null model. This is consistent with the two-level picture that emerges across Sections 3 and 4: the diurnal cycle is a genuine and strong signal in aggregate fraud rate, but at the individual-transaction level it is largely redundant with the discriminative information already contained in the PCA-derived features, which is itself consistent with the thesis's broader argument that ULB's structural conditions limit the practical value of exploiting temporal structure directly.

4. Time Variable: 24-Hour Reset / Diurnal Folding

This section addresses a related but distinct question: whether `Time`, when folded onto a 24-hour clock (`Time %% 86400`), shows a diurnal pattern consistent across both days of the observation window, as opposed to the aggregate rate-based test in Section 3.

4.1. Compute Time-of-Day

```
## Step 1: Fold Time onto a 24-hour clock and identify the day number
fraud_data <- fraud_data %>%
  mutate(
    time_of_day_sec = Time %% 86400,
    time_of_day_hr  = time_of_day_sec / 3600,
    day_number      = floor(Time / 86400) + 1
  )

table(fraud_data$day_number)
```

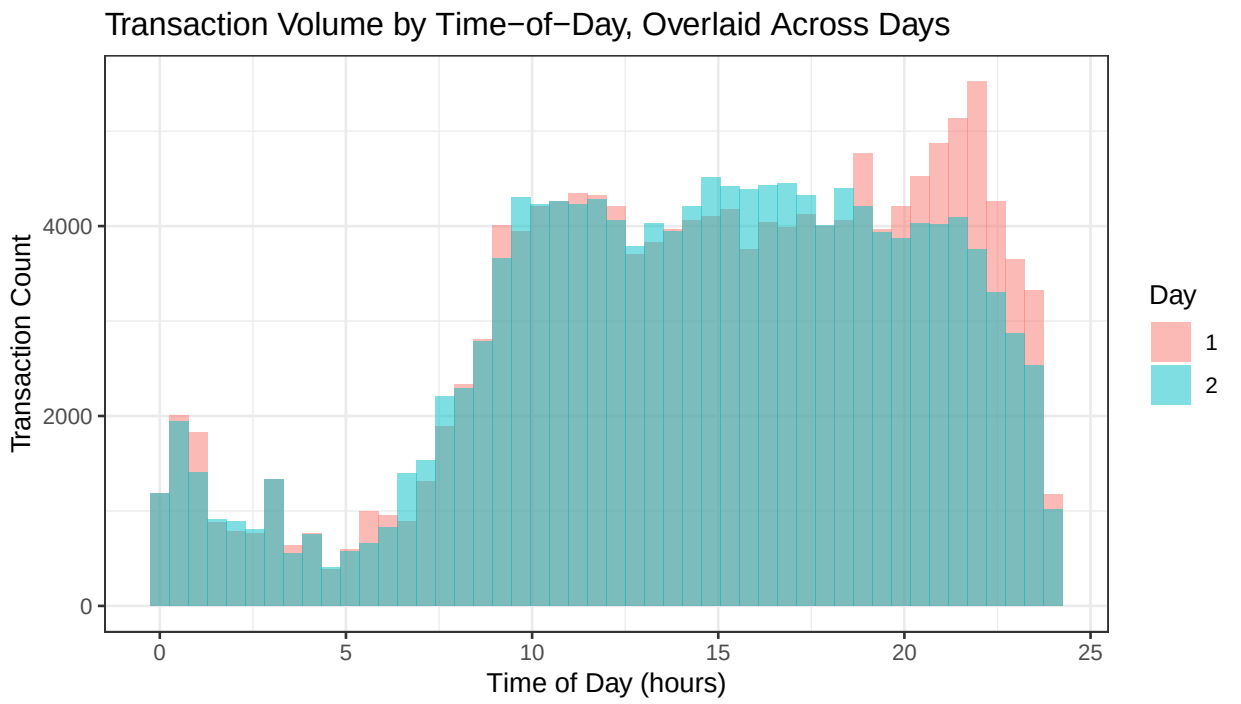
```
##
##      1      2
## 144786 140021
```

```
range(fraud_data$time_of_day_hr)
```

```
## [1]  0.00000 23.99944
```

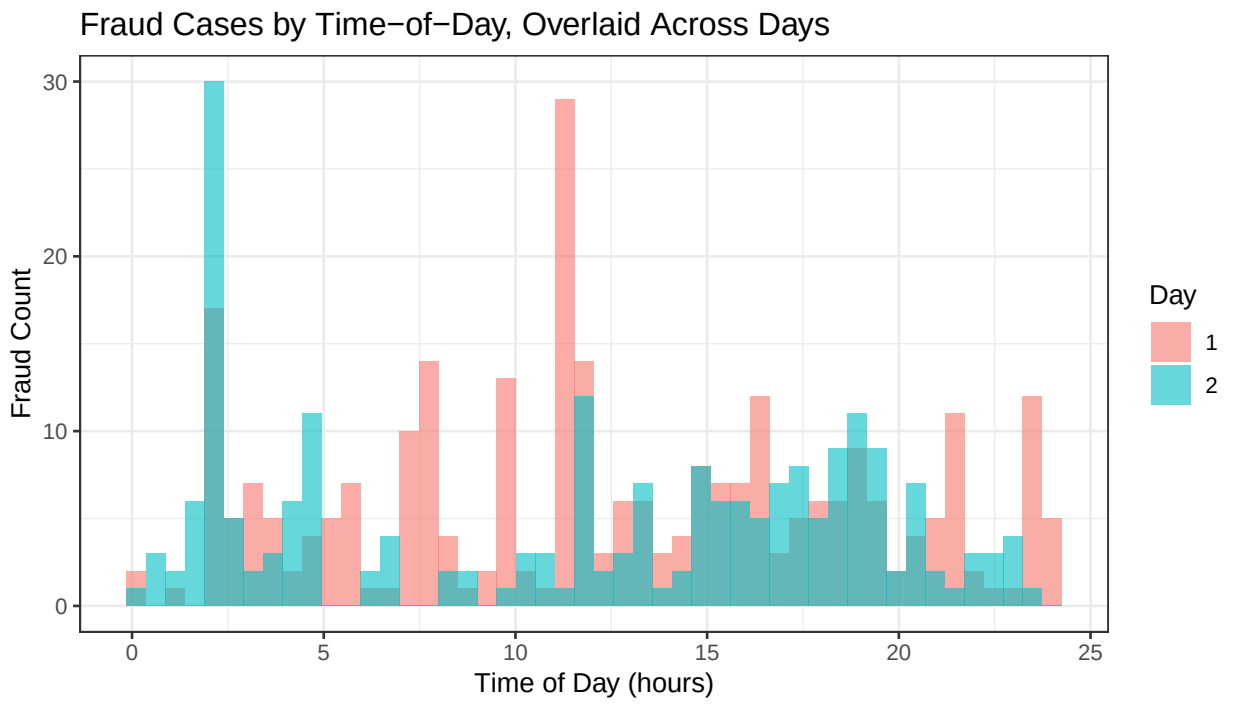
4.2. Overlay Day 1 and Day 2 - All Transactions

```
ggplot(fraud_data, aes(x = time_of_day_hr, fill = factor(day_number))) +
  geom_histogram(bins = 48, position = "identity", alpha = 0.5) +
  labs(
    title = "Transaction Volume by Time-of-Day, Overlaid Across Days",
    x = "Time of Day (hours)",
    y = "Transaction Count",
    fill = "Day"
  ) +
  theme_bw()
```



4.3. Overlay Day 1 and Day 2 - Fraud Cases Only

```
ggplot(fraud_data[fraud_data$Class == 1, ], aes(x = time_of_day_hr, fill = factor(day_number))) +
  geom_histogram(bins = 48, position = "identity", alpha = 0.6) +
  labs(
    title = "Fraud Cases by Time-of-Day, Overlaid Across Days",
    x = "Time of Day (hours)",
    y = "Fraud Count",
    fill = "Day"
  ) +
  theme_bw()
```



4.4. Fraud Rate by Time-of-Day (Both Days Combined)

```
## Step 1: Bin time-of-day into hourly windows and compute rate
time_of_day_summary <- fraud_data %>%
  mutate(hour_bin = floor(time_of_day_hr)) %>%
  group_by(hour_bin) %>%
  summarise(
    fraud_count    = sum(Class),
    n_transactions = n(),
    fraud_rate     = fraud_count / n_transactions,
    .groups = "drop"
  )

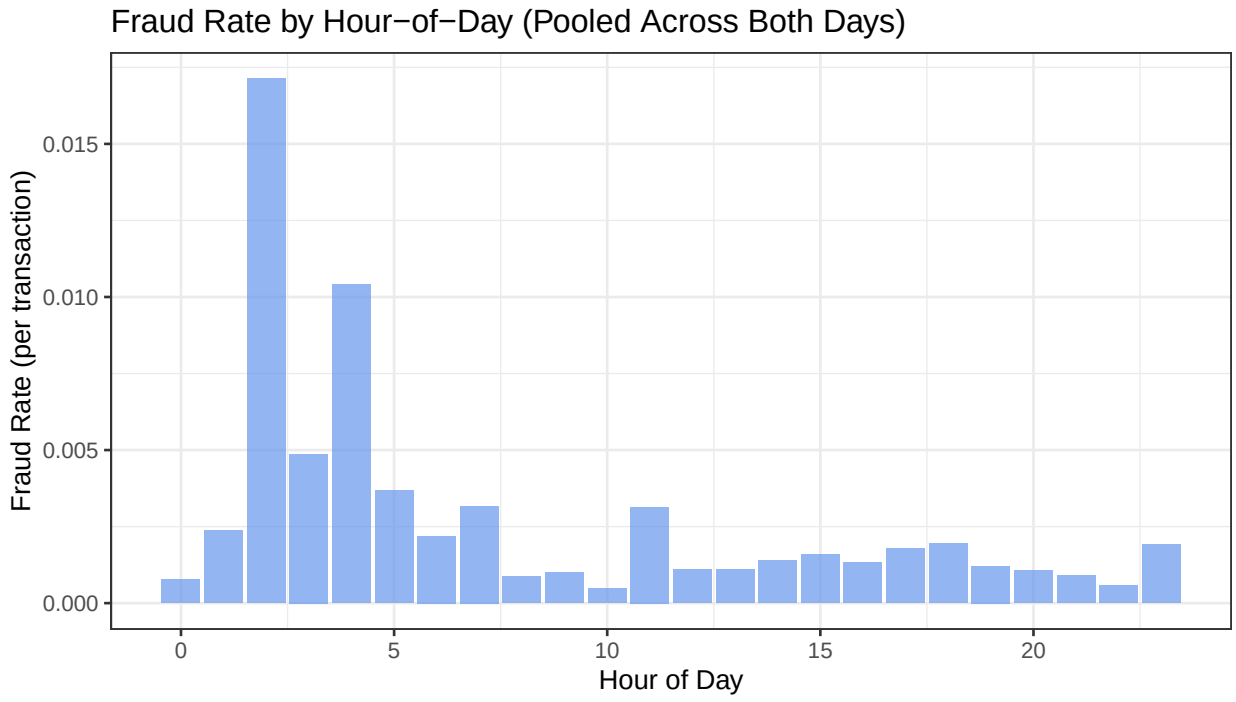
pander(time_of_day_summary, caption = "Fraud Rate by Hour-of-Day (Days 1 and 2 Combined)")
```

Table 2: Fraud Rate by Hour-of-Day (Days 1 and 2 Combined)

hour_bin	fraud_count	n_transactions	fraud_rate
0	6	7695	0.0007797
1	10	4220	0.00237
2	57	3328	0.01713
3	17	3492	0.004868
4	23	2209	0.01041
5	11	2990	0.003679
6	9	4101	0.002195
7	23	7243	0.003175
8	9	10276	0.0008758
9	16	15838	0.00101
10	8	16598	0.000482
11	53	16856	0.003144

hour_bin	fraud_count	n_transactions	fraud_rate
12	17	15420	0.001102
13	17	15365	0.001106
14	23	16570	0.001388
15	26	16461	0.001579
16	22	16453	0.001337
17	29	16166	0.001794
18	33	17039	0.001937
19	19	15649	0.001214
20	18	16756	0.001074
21	16	17703	0.0009038
22	9	15441	0.0005829
23	21	10938	0.00192

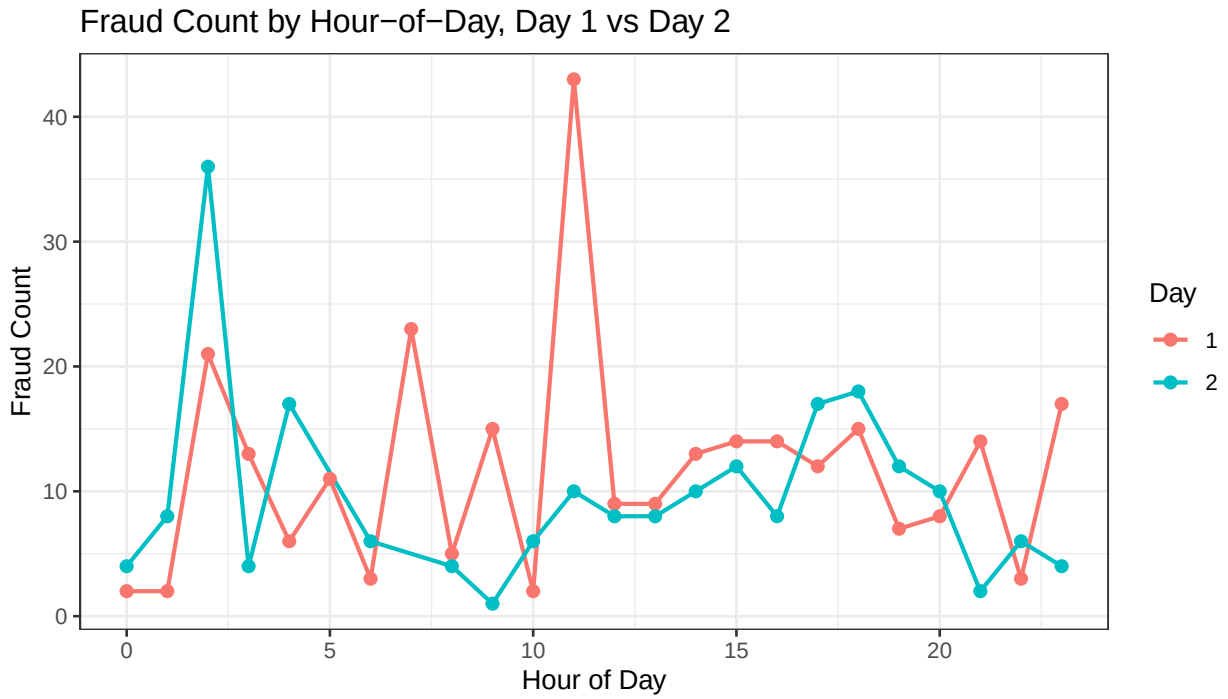
```
ggplot(time_of_day_summary, aes(x = hour_bin, y = fraud_rate)) +
  geom_col(fill = "cornflowerblue", alpha = 0.7) +
  labs(
    title = "Fraud Rate by Hour-of-Day (Pooled Across Both Days)",
    x = "Hour of Day",
    y = "Fraud Rate (per transaction)"
  ) +
  theme_bw()
```



4.5. Day-to-Day Consistency Check

```
fraud_data %>%
  filter(Class == 1) %>%
  mutate(hour_bin = floor(time_of_day_hr)) %>%
  count(day_number, hour_bin) %>%
```

```r
ggplot(aes(x = hour_bin, y = n, colour = factor(day_number))) +
geom_line(linewidth = 0.8) +
geom_point(size = 2) +
labs(
  title = "Fraud Count by Hour-of-Day, Day 1 vs Day 2",
  x = "Hour of Day",
  y = "Fraud Count",
  colour = "Day"
) +
theme_bw()
```



Observations: Transaction volume follows a consistent human-activity pattern across both days (Section 4.2), providing a reasonable baseline for comparing fraud patterns across the two days. The pooled fraud rate by hour-of-day (Section 4.4) shows a sharp peak at hour 2 and a secondary elevation at hour 4, consistent with the diurnal peak identified independently by the harmonic regression in Section 3.7. The day-to-day consistency check (Section 4.5) confirms that the hour-2 elevation recurs on both days (21 cases on Day 1, 36 on Day 2), distinguishing it as a genuine recurring pattern rather than a one-off event; by contrast, the large raw count at hour 11 on Day 1 (43 cases, versus approximately 10 on Day 2) is a single-day anomaly that is substantially diluted once normalised by the much higher transaction volume at that hour, and does not appear as a peak in the pooled rate view.

5. Synthesis

Three independent approaches now converge on the same finding: relative fraud-risk density (main notebook, Section 6.6), harmonic Negative Binomial regression on fraud rate (Section 3 of this notebook), and simple hour-of-day rate binning with a day-to-day consistency check (Section 4 of this notebook) all identify an elevated fraud rate concentrated in the early-morning hours (approximately 2-4am), consistent across both days of the observation window.

This finding does not contradict the thesis's central null result regarding RSF versus Logistic Regression. The temporal structure identified here is aggregate and cyclical, most plausibly driven by reduced transaction

volume and reduced monitoring overnight, rather than genuine instance-level time-to-event structure of the kind RSF's survival framework is designed to exploit. If anything, this sharpens the explanation for the null result: `Time` in the ULB dataset carries some real temporal signal, but of a form that a rate-based or classification-based model is well positioned to capture, while RSF's risk-accumulation framework offers no additional advantage in exploiting it.

The individual-level extension in Sections 3.9-3.10, incorporating the full feature set alongside the harmonic time terms, reinforces this interpretation from a second angle. Once V1-V28 and Amount are included, the diurnal cycle's contribution becomes marginal at best (joint significance test p approximately 0.05), and overall discrimination under the modified Poisson specification (AUPRC 0.630) falls below the full-dataset Logistic Regression benchmark already reported in the main notebook (AUPRC 0.760-0.769). Together, this suggests that while the diurnal cycle is a real and detectable pattern in aggregate fraud rate, it carries little additional information beyond what the PCA-derived features already capture at the transaction level, and does not represent an exploitable structural advantage for either the Poisson framework tested here or, by extension, the RSF framework examined in the main thesis comparison.